

CONTENTS

Algoritmos Ávidos (Greedy)

Guía de estudio con tolerancia cero — Módulo 4, Programación 3 (FING UdelaR) — Capítulo 4, Kleinberg & Tardos

0. Por qué existe este tema

1. Qué es (y qué NO es) un algoritmo ávido

2. Los dos esquemas de demostración de correctitud

2.a) Greedy stays ahead (“se mantiene adelante”)

2.b) Exchange argument (argumento de intercambio)

3. Ejemplo con “greedy stays ahead”: Interval Scheduling (selección de actividades)

Demostración por “greedy stays ahead”

4. Ejemplo con “exchange argument”: Minimizar el atraso máximo (Earliest Deadline First)

Demostración por argumento de intercambio

5. Árboles de expansión mínima: Prim, Kruskal y la propiedad de corte

La propiedad de corte (cut property)

6. Mochila fraccionaria (Fractional Knapsack)

Demostración por argumento de intercambio

7. Trampas típicas de examen

Resumen ultra-rápido (repaso de examen)

Algoritmos Ávidos (Greedy)

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 4, PROGRAMACIÓN 3 (FING UDELAR) — CAPÍTULO 4, KLEINBERG & TARDOS

0. Por qué existe este tema

Casi todos los algoritmos que vimos en Grafos (Módulo 3) —BFS, DFS, Kahn— tienen una propiedad tranquilizadora: **no hay que “elegir” nada**. Se procesa todo lo que hay que procesar, y listo. Los algoritmos ávidos son distintos: en cada paso hay **una decisión real** que tomar (¿qué intervalo agrego? ¿qué arista uso? ¿qué objeto meto en la mochila?), esa decisión se toma mirando **solo la información disponible en ese momento** (criterio *local*, “greedy” = “codicioso”), y **nunca se reconsidera** una vez tomada.

Esto es extremadamente atractivo porque produce algoritmos simples y rápidos. También es la razón por la que este módulo se examina tanto: **la mayoría de las estrategias ávidas “razonables” que a uno se le ocurren NO son óptimas**, y la única forma de saber si la que elegiste sí lo es —sin caer en el peor error posible en un examen de este curso— es tener una **demostración formal** con uno de los dos esquemas que trae el capítulo 4 del libro: *greedy stays ahead* (se mantiene adelante) o *exchange argument* (argumento de intercambio).

Regla de oro de todo el módulo: **ningún algoritmo ávido se acepta como correcto en un examen sin una demostración formal**. “Se ve que funciona” o “es intuitivo que sea óptimo” vale **cero puntos** en la parte de corrección — de hecho, es precisamente el tipo de argumento que casi siempre resulta ser falso (ver sección 7). El capítulo 4 completo es, en el fondo, una caja de herramientas de dos demostraciones (stay-ahead y exchange) aplicadas una y otra vez a problemas distintos.

1. Qué es (y qué NO es) un algoritmo ávido

Definición informal. Un algoritmo ávido resuelve un problema de optimización (maximizar o minimizar alguna cantidad sujeta a restricciones) construyendo la solución **paso a paso**, y en cada paso elige la opción que se ve mejor **según algún criterio local** —sin mirar el futuro, sin backtracking, sin volver nunca sobre una decisión ya tomada.

Formato típico de un problema donde greedy puede (o no) aplicar, según el handout *Guide to Greedy Algorithms* (Roughgarden/Sharp/Wexler, usado como referencia en el curso):

Maximizar/minimizar [cantidad], sujeto a [restricciones].

Ejemplos: “maximizar la cantidad de eventos a los que asisto, sin que se superpongan”; “minimizar el costo de las aristas elegidas, sin desconectar el grafo”.

Lo crucial: que un criterio local “suene razonable” no implica que produzca el óptimo global. La codicia miope puede gastar un recurso barato ahora y quedarse sin nada mejor después.

Ejemplo 1.1 — Contraejemplo clásico: cambio de monedas con denominaciones {1, 3, 4}.

Queremos formar el valor **6** usando la menor cantidad posible de monedas de denominaciones {1, 3, 4} (se pueden repetir).

Estrategia ávida “obvia”: en cada paso, usar la moneda de mayor denominación que no me pase del valor restante.

- Resto=6 → tomo 4 (la mayor que no excede 6) → resto=2.

- Resto=2 → tomo 1 (4 y 3 exceden) → resto=1.

- Resto=1 → tomo 1 → resto=0.

Total greedy: **4+1+1 = 3 monedas**.

Solución óptima real: 3+3 = 6, con solo **2 monedas**.

Verificación (fuerza bruta / programación dinámica, ejecutada en Python): se calculó el mínimo real de monedas para el valor 6 con denominaciones {1,3,4} mediante DP exacta — el óptimo es 2 monedas (3+3), mientras que el algoritmo ávido por denominación descendente entrega 3 monedas (4+1+1).

Confirmado: `greedy=[4,1,1]` (3 monedas) vs `óptimo=[3,3]` (2 monedas).

Este contraejemplo es importante porque el **mismo** esquema greedy (“tomar la moneda más grande posible”) sí es óptimo para las denominaciones de uso real (1,2,5,10,20,50,100,...) — pero eso es una propiedad particular de ESE conjunto de denominaciones, no una propiedad general de la estrategia. En el examen, si aparece un problema de “cambio de monedas” con denominaciones arbitrarias (no necesariamente las de curso de moneda real), **nunca asumas que greedy funciona sin demostrarlo**: para denominaciones arbitrarias el problema de cambio mínimo es, en general, un problema de programación dinámica, no ávido.

2. Los dos esquemas de demostración de correctitud

Cuando un algoritmo ávido produce una solución **factible** (que respeta las restricciones) hay que probarlo aparte —usualmente por inducción simple, mostrando que la elección en cada paso no viola ninguna restricción—, pero la parte que realmente se examina es la prueba de **optimalidad**. El capítulo 4 usa sistemáticamente dos esquemas.

2.A) GREEDY STAYS AHEAD (“SE MANTIENE ADELANTE”)

Idea. Se define una sucesión de mediciones sobre la solución ávida $X = (x_1, x_2, \dots)$ y sobre una solución óptima arbitraria $X^* = (x_1^*, x_2^*, \dots)$, y se demuestra por inducción que, para cada paso i , la medición de la solución ávida es *al menos tan buena* como la de la óptima: $m_i(X) \geq m_i(X^*)$ (o $\leq$, según el sentido del problema). Una vez establecido esto, se concluye la optimalidad —típicamente por absurdo: si la ávida no fuera óptima, el hecho de “mantenerse adelante” da una contradicción directa (por ejemplo, “si necesitara más pasos que la óptima, en el paso en que la óptima ya terminó, la ávida —que iba adelante— también debería haber terminado”).

Estructura recomendada en 4 partes (ver Guide to Greedy Algorithms, CS161):

1. Definir la solución ávida X y una solución óptima arbitraria X^* .
2. Definir la(s) medición(es) $m_i(\cdot)$ que se van a comparar, en una escala que tenga sentido tanto para X como para X^* (esto suele ser el punto más difícil).
3. Demostrar por inducción que greedy se mantiene adelante: $m_i(X) \geq m_i(X^*)$ para todo i .
4. Usar ese hecho para concluir que X es óptima (usualmente por absurdo).

¿Cuándo conviene usar stay-ahead? Es especialmente útil cuando la solución ávida y la óptima pueden tener **distinto “tamaño”** (distinta cantidad de elementos) — por ejemplo, cuando el algoritmo va agregando elementos uno a uno hasta que “no hace falta más” (cobertura de intervalos, selección de actividades). Ahí “mantenerse adelante” permite razonar: “si la óptima ya no necesitó más pasos en el paso k , la ávida —que iba al menos tan bien— tampoco los necesita”, acotando el tamaño de la solución ávida.

2.B) EXCHANGE ARGUMENT (ARGUMENTO DE INTERCAMBIO)

Idea. Se toma una solución óptima arbitraria X^* y se demuestra que, si $X^* \neq X$ (la solución ávida), entonces X^* se puede **transformar** en una solución que se parece más a X (intercambiando una “pieza” de X^* por la pieza correspondiente de X) **sin empeorar su costo**. Iterando este intercambio una cantidad finita de veces, X^* se transforma en X sin perder optimalidad en el camino — por lo tanto X también es óptima.

Estructura recomendada en 4 partes:

1. Definir X (ávida) y X^* (óptima arbitraria).
2. Si $X \neq X^*$, mostrar que deben diferir en algo concreto (una pieza de X que no está en X^* , o dos elementos en orden distinto) y darle nombre a esa diferencia.
3. Mostrar el intercambio: reemplazar esa pieza de X^* por la correspondiente de X , y probar que el costo/valor de X^* no empeora (por lo tanto la nueva solución también es óptima).
4. Argumentar que el intercambio reduce estrictamente el número de diferencias entre X y X^* , así que iterando se llega a $X^* = X$, y por lo tanto X es óptima.

¿Cuándo conviene usar exchange argument? Funciona muy bien cuando **todas** las soluciones factibles tienen el mismo “tamaño” o “forma” y solo difieren en el **orden** o en la **composición interna** — por ejemplo, todos los *schedules* tienen los mismos n trabajos, solo en distinto orden (scheduling), o todos los árboles de expansión tienen exactamente $n-1$ aristas (MST). Ahí “intercambiar una pieza por otra del mismo tipo” tiene sentido directo. Es, en general, más simple de escribir que stay-ahead cuando aplica, porque no hay que inventar una escala de medición absoluta.

Truco de examen: antes de intentar demostrar que “greedy se mantiene adelante” según tu medición elegida, probá primero la implicación al revés: “si greedy se mantiene adelante según esta medición, ¿de ahí sale la optimalidad?”. Si no sale, cambiá la medición ANTES de gastar tiempo demostrando una inducción que no te va a servir. Es el error más común y el que más tiempo hace perder en un parcial.

⚠ **Error de tolerancia cero:** escribir “por inducción, en cada paso greedy es al menos tan bueno como cualquier otra solución, luego greedy es óptimo” SIN definir explícitamente qué se está midiendo (m_i) ni demostrar el paso inductivo con el caso base y el paso general por separado. Esa frase, sin la maquinaria explícita, no es una demostración — es una afirmación de que existe una demostración, que es exactamente lo que hay que escribir.

3. Ejemplo con “greedy stays ahead”: Interval Scheduling (selección de actividades)

Problema. Dado un conjunto de intervalos (actividades) con hora de inicio s_i y hora de fin f_i , seleccionar el subconjunto más grande de intervalos **mutuamente compatibles** (que no se superpongan: dos intervalos i, j son compatibles si $f_i \leq s_j$ o $f_j \leq s_i$).

Algoritmo ávido (ordenar por hora de fin):

```
IntervalScheduling(intervalos):
    ordenar los intervalos por f_i creciente
    A = ∅          # conjunto de intervalos aceptados
    fin_actual = -∞
    para cada intervalo i en orden de f_i creciente:
        si s_i ≥ fin_actual:
            agregar i a A
            fin_actual = f_i
    devolver A
```

Complejidad: $O(n \log n)$ por el ordenamiento; el recorrido posterior es $O(n)$. Total **$O(n \log n)$** .

DEMOSTRACIÓN POR “GREEDY STAYS AHEAD”

Sea $A = \{i_1, i_2, \dots, i_k\}$ la solución que produce el algoritmo, indexada en el orden en que fueron aceptados (que coincide con el orden creciente de sus horas de fin: $f(i_1) < f(i_2) < \dots < f(i_k)$). Sea $O = \{j_1, j_2, \dots, j_m\}$ **cualquier** solución compatible, también indexada por hora de fin creciente.

Medición: $m_r = f(i_r)$ para la solución ávida, $m_r^* = f(j_r)$ para la solución O — es decir, comparamos, para cada r , la hora de fin del r -ésimo evento aceptado en cada solución.

Lema (greedy se mantiene adelante). Para todo r tal que $1 \leq r \leq k$, si O tiene al menos r elementos, entonces $f(i_r) \leq f(j_r)$.

Demostración por inducción en r .

- **Caso base ($r=1$):** el algoritmo ávido elige, entre todos los intervalos, el que tiene menor hora de fin. Por lo tanto $f(i_1) \leq f(j_r)$ para cualquier intervalo j_r posible, en particular $f(i_1) \leq f(j_1)$. ✓

- **Paso inductivo:** supongamos $f(i_r) \leq f(j_r)$ (hipótesis inductiva) y que O tiene al menos $r+1$ elementos. Como O es una solución factible, j_{r+1} es compatible con j_r , así que $s(j_{r+1}) \geq f(j_r) \geq f(i_r)$ (esta última desigualdad es la hipótesis inductiva). Es decir, j_{r+1} es un intervalo que empieza después de $f(i_r)$ — por lo tanto, en el momento en que el algoritmo ávido terminó de procesar i_r , el intervalo j_{r+1} era una opción **disponible** (compatible con lo ya elegido, $\text{fin_actual} = f(i_r) \leq s(j_{r+1})$). Como el algoritmo ávido, entre todos los intervalos disponibles en ese momento, elige el de menor hora de fin para ser i_{r+1} , se cumple $f(i_{r+1}) \leq f(j_{r+1})$.

■

Corolario de optimalidad. Supongamos, por absurdo, que existe una solución óptima O con $|O| = m > k = |A|$. Por el lema, en particular para $r = k$ se cumple $f(i_k) \leq f(j_k)$. Como O tiene al menos $k+1$ elementos, j_{k+1} existe y es compatible con j_k , así que $s(j_{k+1}) \geq f(j_k) \geq f(i_k) = \text{fin_actual}$ del algoritmo ávido al terminar. Pero entonces j_{k+1} era un intervalo disponible que el algoritmo ávido no tomó a pesar de que el algoritmo continúa mientras haya intervalos disponibles — contradicción con la definición del algoritmo (el ávido termina solo cuando no queda ningún intervalo compatible con fin_actual). Por lo tanto $m \leq k$, y como A es factible y O es óptima, $k \leq m$. Luego $k = m$: **A es óptima.** ■

Ejemplo numérico verificado (8 intervalos).

Intervalo	Inicio	Fin
a	1	4
b	3	5
c	0	6
d	5	7
e	3	8
f	5	9
g	6	10
h	8	11

Orden por hora de fin: a(4), b(5), c(6), d(7), e(8), f(9), g(10), h(11).

Traza del algoritmo: $\text{fin_actual} = -\infty$.

- $a=[1,4]$: $1 \geq -\infty \rightarrow$ acepto. $\text{fin_actual}=4$.
- $b=[3,5]$: $3 < 4 \rightarrow$ rechazo (se superpone con a).
- $c=[0,6]$: $0 < 4 \rightarrow$ rechazo.
- $d=[5,7]$: $5 \geq 4 \rightarrow$ acepto. $\text{fin_actual}=7$.
- $e=[3,8]$: $3 < 7 \rightarrow$ rechazo.
- $f=[5,9]$: $5 < 7 \rightarrow$ rechazo.
- $g=[6,10]$: $6 < 7 \rightarrow$ rechazo.
- $h=[8,11]$: $8 \geq 7 \rightarrow$ acepto. $\text{fin_actual}=11$.

Solución ávida: {a, d, h}, tamaño 3.

Verificación por fuerza bruta (script Python, $2^8=256$ subconjuntos evaluados, se probó compatibilidad par a par en cada uno): el subconjunto compatible más grande tiene tamaño **3**, y uno de los óptimos encontrados es exactamente {a, d, h}. Coincide con el resultado del algoritmo ávido. ✓

4. Ejemplo con “exchange argument”: Minimizar el atraso máximo (Earliest Deadline First)

Problema. Hay n trabajos, cada uno con un tiempo de procesamiento t_i y una fecha límite (deadline) d_i . Se procesan de a uno, sin interrupción, en una única máquina, comenzando en el tiempo 0. Si el trabajo i termina en el tiempo $f(i)$, su **atraso** (lateness) es $\ell_i = \max(0, f(i) - d_i)$ (en la formulación del libro, se suele definir directamente $\ell_i = f(i) - d_i$, permitiendo valores negativos cuando termina antes del plazo). Se busca el orden que **minimiza el atraso máximo** $L = \max_i \ell_i$.

Algoritmo ávido (Earliest Deadline First, EDF): procesar los trabajos en orden creciente de deadline d_i , sin dejar huecos de inactividad.

```
EDF(trabajos):
    ordenar los trabajos por d_i creciente
    tiempo = 0
    para cada trabajo i en ese orden:
        programar i en [tiempo, tiempo + t_i]
        tiempo = tiempo + t_i
```

Complejidad: $O(n \log n)$ (ordenamiento) + $O(n)$ (asignación) = **$O(n \log n)$** .

DEMOSTRACIÓN POR ARGUMENTO DE INTERCAMBIO

Paso 1 — factibilidad / definición. Un schedule sin tiempos muertos (sin huecos de inactividad entre trabajos) queda determinado únicamente por el **orden** de los trabajos. Todo schedule óptimo puede asumirse sin tiempos muertos (agregar tiempo ocioso nunca reduce el atraso máximo). Así que comparamos únicamente órdenes.

Paso 2 — inversión. Decimos que un schedule tiene una **inversión** si existen trabajos i, j tales que i se procesa inmediatamente antes que j , pero $d_j < d_i$ (j tiene un deadline más urgente pero se atiende después). El schedule EDF, por construcción, **no tiene inversiones** (procesa siempre en orden de deadline creciente).

Paso 3 — intercambiar para eliminar una inversión no empeora el atraso máximo. Sea S un schedule con una inversión entre un par **adyacente** i, j (i inmediatamente antes de j , $d_j < d_i$). Sea S' el schedule que resulta de intercambiar i y j (y dejar todo lo demás igual). Como i y j son adyacentes, el intervalo de tiempo que ocupan juntos es el mismo en S y en S' ; solo cambia el orden interno.

- Sea f el tiempo en que termina i en S (que es el mismo tiempo en que termina j en S'), y sea $t = t_i + t_j$.
- En S : i termina en f , con atraso $\ell_i = f - t_j - d_i$; j termina en $f + t_j$ (o sea, en f), con atraso $\ell_j = f - d_j$. (Aquí “ f ” denota el instante en que termina el par en S , es decir el fin de j .)

- Todos los demás trabajos (fuera del par i, j) terminan exactamente en el mismo instante en S y en S' , así que sus atrasos no cambian.
- Dentro del par, en S' j termina en $f - t_i$ y luego i termina en f (el instante final del par no cambia). El atraso de j en S' es $\ell'_j = (f - t_i) - d_j$, y como $t_i > 0$, $\ell'_j < \ell_j$ (mejora). El atraso de i en S' es $\ell'_i = f - d_i$.
- Como $d_j < d_i$ (hipótesis de inversión), $\ell'_i = f - d_i < f - d_j = \ell_j$ (el nuevo atraso de i es menor que el atraso QUE TENÍA j en S).

Juntando: $\ell'_i < \ell_j$, y $\ell'_j < \ell_j$, así que $\max(\ell'_i, \ell'_j) \leq \ell_j = \max(\ell_i, \ell_j)$ (ya que en S , con $d_j < d_i$, se tiene $\ell_j \geq \ell_i$ — el trabajo con deadline más urgente que además se atiende después nunca puede tener un atraso menor que el otro cuando se calculan con el mismo f , dado que ambos terminan en el mismo instante f y $d_j < d_i$ implica $f - d_j > f - d_i$). Por lo tanto **el atraso máximo del par no aumenta al intercambiar**, y como el resto de los trabajos no cambia su atraso, **el atraso máximo global de S' es $\leq$ al de S .**

Paso 4 — iterar. Cada intercambio de un par adyacente invertido elimina esa inversión y no crea inversiones nuevas entre otros pares (solo reordena i, j entre sí). El número de inversiones es finito (a lo sumo $C(n, 2)$) y cada intercambio lo reduce estrictamente en al menos 1. Iterando, se llega en una cantidad finita de pasos a un schedule **sin ninguna inversión** —es decir, ordenado por deadline creciente, exactamente el orden que produce EDF— sin haber aumentado el atraso máximo en ningún paso. Por lo tanto, **el atraso máximo de EDF es $\leq$ al atraso máximo de cualquier schedule óptimo**, y como EDF es un schedule factible, **EDF es óptimo. ■**

Ejemplo numérico verificado (6 trabajos).

Trabajo	t_i (duración)	d_i (deadline)
j1	3	6
j2	2	8
j3	1	9
j4	4	9
j5	3	14
j6	2	15

Orden EDF (ya están ordenados por d_i creciente: 6,8,9,9,14,15 — con empate j3/j4 en $d=9$, cualquier orden entre ellos es válido, se ilustra con j3 antes de j4):

Orden	Inicio	Fin	d_i	Atraso ($\text{fin}-d_i$)
j1	0	3	6	-3
j2	3	5	8	-3
j3	5	6	9	-3
j4	6	10	9	+1
j5	10	13	14	-1
j6	13	15	15	0

Atraso máximo EDF = 1 (lo produce j4).

Verificación por fuerza bruta (script Python, se probaron las $6! = 720$ permutaciones posibles de los 6 trabajos, calculando el atraso máximo de cada una): el atraso máximo óptimo sobre las 720 permutaciones es exactamente **1**, alcanzado (entre otros órdenes) por el orden EDF j1,j2,j3,j4,j5,j6. Coincide. ✓

Contraste con una heurística ingenua (para reforzar por qué EDF y no “trabajo más corto primero”): ordenando los mismos 6 trabajos por duración creciente (SPT, shortest processing time first) en vez de por deadline, se obtiene un atraso máximo de **6** (peor que el óptimo, que es 1) — verificado con el mismo script. Confirma que “atender primero lo que se hace más rápido” es un criterio local razonable pero **no óptimo** para este problema (ver también sección 7).

5. Árboles de expansión mínima: Prim, Kruskal y la propiedad de corte

Problema (MST). Dado un grafo conexo $G=(V,E)$ con costos c_e en las aristas (todos distintos, para simplificar), encontrar un árbol de expansión $T \subseteq E$ (subgrafo conexo, acíclico, que toca todos los vértices) de costo total mínimo.

LA PROPIEDAD DE CORTE (CUT PROPERTY)

Un **corte** $(S, V \setminus S)$ es una partición de los vértices en dos conjuntos no vacíos. Una arista **cruza** el corte si tiene un extremo en cada lado.

Propiedad del corte (Cut Property, enunciado 4.20 del libro): supongamos que todos los costos de arista son distintos. Sea $(S, V \setminus S)$ cualquier corte del grafo, y sea e la arista de **menor costo** entre las que cruzan ese corte. Entonces **toda** MST del grafo contiene a e .

Idea de la demostración (exchange argument): si un MST T no contuviera a $e=(v,w)$, agregar e a T genera un ciclo, y ese ciclo debe cruzar el corte $(S, V \setminus S)$ al menos dos veces (una para “salir” y otra para “volver”) — es decir, hay otra arista e' en T que también cruza el corte. Como $c_e < c_{e'}$ (e es la de menor costo que cruza el corte, y los costos son todos distintos), reemplazar e' por e en T da un árbol de expansión de menor costo, contradiciendo que T era mínimo.

Esta única propiedad es la que garantiza la correctitud de **ambos** algoritmos clásicos:

- **Prim:** en cada paso, S es el conjunto de vértices ya conectados por el árbol parcial; el algoritmo agrega la arista de menor costo que cruza el corte $(S, V \setminus S)$. Por la propiedad de corte, esa arista pertenece a algún MST — y como nunca se elimina una arista ya agregada, al final T es exactamente un MST.
- **Kruskal:** se recorren las aristas en orden creciente de costo, agregando una arista si no forma ciclo con las ya elegidas. Cuando Kruskal agrega una arista $e=(u,v)$, los conjuntos conexos (componentes) que contienen a u y a v en ese momento forman un corte $(S, V \setminus S)$ donde S es la componente de u ; como e es la arista de menor costo procesada hasta ahora y no forma ciclo, e es la arista de menor costo que cruza ese corte (cualquier arista más barata que cruzara el corte ya habría sido considerada antes, y si no se agregó es porque unía dos vértices ya en la misma componente, es decir no cruzaba ESE corte). Por la propiedad de corte, e pertenece a algún MST.

Ejemplo numérico verificado — mismo grafo para Prim y Kruskal.

Grafo con 6 nodos $\{A,B,C,D,E,F\}$ y aristas: $A-B(4)$, $A-C(2)$, $B-C(1)$, $B-D(5)$, $C-D(8)$, $C-E(10)$, $D-E(2)$, $D-F(6)$, $E-F(3)$.

Kruskal (orden creciente de costo: $B-C(1)$, $A-C(2)$, $D-E(2)$, $E-F(3)$, $A-B(4)$, $B-D(5)$, $D-F(6)$, $C-D(8)$, $C-E(10)$):

- $B-C(1)$: no forma ciclo $\rightarrow$ acepto.
- $A-C(2)$: no forma ciclo $\rightarrow$ acepto.
- $D-E(2)$: no forma ciclo $\rightarrow$ acepto.
- $E-F(3)$: no forma ciclo $\rightarrow$ acepto.
- $A-B(4)$: A y B ya conectados (vía C) $\rightarrow$ forma ciclo $\rightarrow$ rechazo.
- $B-D(5)$: B y D en componentes distintas $\rightarrow$ acepto. (ya van 5 aristas = $n-1$, se detiene)

MST Kruskal = $\{B-C(1), A-C(2), D-E(2), E-F(3), B-D(5)\}$, **costo total = 13**.

Prim (arrancando en A): agrega en orden $A-C(2)$ [corte $\{A\}$ vs resto, mínimo es $A-C=2$], luego $C-B(1)$ [corte $\{A,C\}$ vs resto, mínimo es $B-C=1$], luego $B-D(5)$ [corte $\{A,B,C\}$ vs resto: candidatas $B-D=5$, $C-D=8$, $C-E=10 \rightarrow$ mínimo $B-D=5$], luego $D-E(2)$ [corte $\{A,B,C,D\}$ vs resto: candidatas $D-E=2$, $D-F=6$, $C-E=10 \rightarrow$ mínimo $D-E=2$], luego $E-F(3)$ [corte $\{A,B,C,D,E\}$ vs $\{F\}$: candidatas $D-F=6$, $E-F=3 \rightarrow$ mínimo $E-F=3$].

MST Prim = $\{A-C(2), B-C(1), B-D(5), D-E(2), E-F(3)\}$, **costo total = 13**.

Verificación: ambos árboles tienen el mismo conjunto de aristas (coinciden exactamente en este caso, como debe ocurrir cuando todos los costos son distintos — sección 6.1 del práctico) y el mismo costo total = 13. Se verificó además por **fuerza bruta** (enumerando los $C(9,5)=126$ subconjuntos de 5 aristas de los 9 disponibles, y quedándose con los que forman árbol de expansión válido) que **13 es efectivamente el costo mínimo posible** sobre todo el grafo. Coincide con Prim y Kruskal. ✓

Como los 9 costos del ejemplo son todos distintos, por el Ejercicio 1 del Práctico 6 (K&T Ex. 4.8, ver también Práctico4_Resolucion.md) el MST es **único** — por eso Prim y Kruskal, partiendo de estrategias distintas, llegan exactamente al mismo árbol.

6. Mochila fraccionaria (Fractional Knapsack)

Problema. Hay n objetos, cada uno con valor p_i y peso w_i , y una mochila de capacidad M . A diferencia de la mochila 0/1, acá se puede tomar **cualquier fracción** $0 \leq x_i \leq 1$ de cada objeto, ganando $x_i \cdot p_i$ de valor y ocupando $x_i \cdot w_i$ de peso. Se busca maximizar $\sum x_i p_i$ sujeto a $\sum x_i w_i \leq M$.

Algoritmo ávido: ordenar los objetos por razón valor/peso (p_i/w_i) **decreciente**, y llenar la mochila tomando cada objeto por completo mientras haya capacidad, y al último objeto que no entra completo tomarle solo la fracción que falta para llenar M .

```
MochilaFraccionaria(objetos, M):
    ordenar objetos por p_i/w_i decreciente
    x = vector de ceros
    resto = M
    para cada objeto i en ese orden:
        si resto == 0: cortar
        tomar = min(w_i, resto)
        x_i = tomar / w_i
        resto = resto - tomar
    devolver x
```

Complejidad: $O(n \log n)$ por el ordenamiento.

DEMOSTRACIÓN POR ARGUMENTO DE INTERCAMBIO

Esta es la estructura que aparece en el material de cátedra “Demostración Correctitud Mochila Greedy” (imágenes 1-3), que se resume y verifica a continuación.

Sea $X = (x_1, \dots, x_n)$ la solución que genera el algoritmo ávido. Por construcción, X tiene la forma $X = (1, \dots, 1, x_j, 0, \dots, 0)$: hay un índice de corte j tal que todos los objetos con índice menor a j (es decir, de mayor razón p/w , ya que están ordenados en forma decreciente) se toman completos, el objeto j se toma parcialmente ($0 \leq x_j < 1$, o $x_j=1$ si justo entra completo y no sobra nada), y los objetos posteriores no se tocan.

Por absurdo: supongamos que X no es óptima, y sea $Y = (y_1, \dots, y_n)$ una solución óptima. Como $X \neq Y$, existe un menor índice k tal que $y_k \neq x_k$. Se demuestra, analizando los tres casos posibles según la posición de k respecto de j ($k < j$, $k=j$, $k > j$) y usando en cada caso la restricción de capacidad $\sum x_i w_i = M$, que necesariamente $y_k < x_k$ — es decir, en el primer punto donde difieren, la solución óptima tiene *menos* del objeto k que la ávida (intuitivamente: como los objetos previos a k ya están saturados a $x=1$ en ambas soluciones, si Y tomara $y_k \geq x_k$ con $x_k < 1$ en algún caso, Y estaría usando más capacidad que la disponible, o comprometiendo objetos posteriores; el análisis completo de los tres casos se muestra en el material de cátedra).

Construcción de Z (el intercambio): se define una nueva solución Z que coincide con X (y por lo tanto con Y) en las posiciones $1..k-1$, aumenta la componente k hasta $z_k = x_k$ (es decir, “sube” y_k hasta el valor que tenía la ávida), y compensa ese aumento de peso **reduciendo** proporcionalmente los componentes posteriores a k (los y_i con $i > k$) lo justo para no exceder la capacidad M: se elige Z tal que $\sum_{i=k+1}^n (y_i - z_i)w_i = (z_k - y_k)w_k$ (el peso que se “libera” atrás es exactamente el que se necesita para el aumento en k).

La ganancia no empeora: al comparar la ganancia de Z contra la de Y,

$$\text{Ganancia}(Z) = \text{Ganancia}(Y) + (z_k - y_k) \cdot p_k - \sum_{i=k+1}^n (y_i - z_i) \cdot p_i$$

y usando que $p_k/w_k \geq p_i/w_i$ para todo $i > k$ (porque los objetos están ordenados por razón decreciente, y $k < i$), se puede acotar cada p_i por $(p_k/w_k) \cdot w_i$ en la resta, obteniendo:

$$\text{Ganancia}(Z) \geq \text{Ganancia}(Y) + (p_k/w_k) \cdot [(z_k - y_k)w_k - \sum_{i=k+1}^n (y_i - z_i)w_i] = \text{Ganancia}(Y) + (p_k/w_k) \cdot 0 = \text{Ganancia}(Y)$$

(el término entre corchetes es exactamente 0 por cómo se construyó Z). Como Y es óptima, Ganancia(Z) no puede ser estrictamente mayor que Ganancia(Y) — así que Ganancia(Z) = Ganancia(Y), es decir **Z es también una solución óptima**, y Z coincide con X en una componente más que Y (coincide hasta la posición k inclusive). Repitiendo este proceso (usando Z en lugar de Y) una cantidad finita de veces —el índice de la primera discrepancia crece estrictamente en cada iteración— se llega a una solución óptima que coincide con X en **todas** las componentes. Por lo tanto **X es óptima**. ■

Ejemplo numérico verificado.

Objetos: A(valor=60, peso=10), B(valor=100, peso=20), C(valor=120, peso=30). Capacidad M=50.

Razones valor/peso: A=6, B=5, C=4 → orden ávido: A, B, C.

Traza: tomo A completo (peso 10, resto 40), tomo B completo (peso 20, resto 20), de C tomo $20/30 = 2/3$ (peso 20, resto 0).

Valor total = $60 + 100 + (2/3)(120) = 60 + 100 + 80 = 240$.

Verificación: se realizó una búsqueda aleatoria de 200.000 asignaciones fraccionarias distintas que respetan la capacidad (muestreo uniforme del simplex de fracciones, reescalado si excedía M) — ninguna superó el valor 240 obtenido por el algoritmo ávido, consistente con que 240 es el óptimo (como corresponde a un problema de programación lineal donde el óptimo se alcanza siempre en un vértice de la región factible, que es exactamente lo que produce la estrategia de “llenar por razón decreciente”). ✓

Contrastar con la sección 7: si el **mismo** problema se plantea como mochila **0/1** (no se permite fraccionar, $x_i \in \{0, 1\}$), el algoritmo ávido por razón valor/peso deja de ser óptimo — ver Ejemplo 7.1.

7. Trampas típicas de examen

⚠ **Trampa 1 — Mochila 0/1: el greedy por razón valor/peso NO es óptimo.**

Mismos objetos que en la sección 6: A(60,10), B(100,20), C(120,30), capacidad M=50, pero ahora **sin fraccionar** (cada objeto se toma completo o no se toma).

Greedy por razón v/w (orden A,B,C): toma A (peso 10, resto 40), toma B (peso 20, resto 20), C no entra completo (peso 30 > resto 20) → se descarta. **Valor greedy = 60+100 = 160.**

Óptimo real (verificado por fuerza bruta sobre las $2^3=8$ combinaciones posibles): tomar {B, C} (peso 20+30=50=M) da valor 100+120=220, que es mejor.

Confirmado con Python: greedy=160, óptimo=220 → el greedy por razón v/w es estrictamente subóptimo en mochila 0/1. La diferencia clave con la sección 6 es que ahí, al no poder fraccionar, “saturar” el objeto de mejor razón puede dejar un resto de capacidad que ningún objeto restante llena bien, mientras que una combinación distinta aprovecha mejor la capacidad total. La mochila 0/1 NO es un problema resoluble por greedy en general — se resuelve por programación dinámica (tema de otro módulo).

⚠ Trampa 2 — Interval Scheduling: ordenar por duración (o por inicio) en vez de por hora de fin.

Instancia: intervalos “largo”=[0,10], a=[0,4], b=[4,8], c=[8,12], “corto”=[3,5].

Óptimo real (fuerza bruta): {a, b, c}, tamaño 3.

Greedy por duración más corta primero (criterio intuitivo: “atender primero lo rápido”): toma “corto”=[3,5] (duración 2, la menor), luego el único compatible que queda es c=[8,12] → tamaño 2. Subóptimo.

Greedy por hora de inicio más temprana (otro criterio intuitivo: “empezar por lo que se puede empezar antes”): toma “largo”=[0,10] (empieza en 0) y ya no hay nada compatible → tamaño 1. Muy subóptimo.

Confirmado con Python sobre esta instancia: fuerza_bruta=3, greedy_duración=2, greedy_inicio=1. Solo ordenar por **hora de fin** (sección 3) da el óptimo 3 en este caso. Cualquier otro criterio de ordenamiento “razonable” (duración, inicio, cantidad de solapamientos) puede fallar, y cada uno tiene su propio contraejemplo.

⚠ Trampa 3 — “Es obvio que funciona” no es una demostración, y a veces ni siquiera es cierto.

El error más común en los parciales de este módulo no es matemático sino de *redacción*: escribir la estrategia ávida correcta pero justificar su optimalidad con una frase intuitiva (“como en cada paso tomamos lo mejor disponible, el resultado global también es el mejor”) en lugar de con uno de los dos esquemas formales (secciones 2, 3, 4). Esa frase es además **lógicamente inválida en general**: las Trampas 1 y 2 de esta sección son precisamente casos donde “tomar lo mejor disponible en cada paso” (mayor razón v/w; evento más corto; evento que empieza antes) suena igual de razonable que las estrategias correctas de las secciones 3, 4 y 6, y sin embargo no son óptimas. La única forma de distinguir, en un examen, una estrategia ávida correcta de una incorrecta es intentar construir la demostración formal — si en el intento de demostrar “greedy se mantiene adelante” o el argumento de intercambio aparece un paso que no se puede justificar, esa es la señal de que la estrategia (probablemente) está mal, y hay que buscar un contraejemplo en lugar de forzar la demostración.

Resumen ultra-rápido (repaso de examen)

- Un algoritmo **greedy** construye la solución paso a paso con un criterio **local**, sin reconsiderar decisiones. Puede ser rápido y simple, pero **casi nunca es obvio que sea óptimo** — hace falta demostrarlo siempre.
- **Contraejemplo de referencia:** cambio de monedas {1,3,4} para el valor 6 → greedy da 3 monedas (4+1+1), óptimo da 2 (3+3). Greedy no es una propiedad universal de “tomar lo más grande primero”.

- **Greedy stays ahead:** definir medición $m_i(X)$ vs $m_i(X^*)$, demostrar por inducción que greedy nunca queda atrás, concluir optimalidad (típicamente por absurdo). Útil cuando las soluciones pueden tener distinto tamaño (Interval Scheduling).
- **Exchange argument:** tomar una óptima X^* arbitraria, mostrar que si difiere de la ávida X se puede intercambiar una pieza sin empeorar el costo, iterar hasta que $X^*=X$. Útil cuando todas las soluciones tienen la misma “forma” (Scheduling/EDF, MST, Mochila fraccionaria).
- **Interval Scheduling:** ordenar por hora de **fin** creciente $\rightarrow O(n \log n)$. Demostración: stay-ahead sobre $f(i_r)$ vs $f(j_r)$.
- **Minimizar lateness máximo (EDF):** ordenar por **deadline** creciente, sin huecos $\rightarrow O(n \log n)$. Demostración: exchange argument eliminando inversiones adyacentes.
- **MST (Prim y Kruskal):** ambos correctos por la **propiedad de corte** (cut property): la arista más barata que cruza cualquier corte pertenece a algún MST. Con costos todos distintos, el MST es único, y Prim/Kruskal coinciden.
- **Mochila fraccionaria:** ordenar por razón valor/peso decreciente, saturar objetos hasta llenar la capacidad $\rightarrow O(n \log n)$. Óptima por exchange argument (a diferencia de la mochila 0/1, que NO es greedy-óptima — Trampa 1).
- **Nunca** se acepta “es obvio que funciona” como demostración de optimalidad — siempre usar uno de los dos esquemas formales de la sección 2.